
TRAINING MATERIAL

React JS

Complete Theory Notes — Basic to Advanced, with Examples

Fork Infosystems

Computer Education Institute, Karad | in association with CodInt Digitech

Companion reference to the 15-Day React JS Training Guide

Table of Contents

Part A — Basics

1. Introduction to React
2. Setting Up a React Project
3. JSX
4. Components
5. Props

Part B — Core Concepts

1. State and the useState Hook
2. Event Handling
3. Conditional Rendering
4. Lists and Keys
5. Forms and Controlled Components

Part C — Intermediate

1. useEffect and Side Effects
2. Component Composition and Lifting State Up
3. React Router
4. Context API

Part D — Advanced

1. Custom Hooks
 2. useReducer
 3. Performance Optimization
 4. Connecting to APIs
 5. Error Boundaries
 6. Code Splitting and Lazy Loading
 7. State Management Beyond React
 8. Deployment and Next Steps
-

PART A — BASICS

1. Introduction to React

React is a JavaScript **library** for building user interfaces, created and maintained by Meta (formerly Facebook). It is not a full framework — it focuses specifically on the "view" layer of an application, and is commonly paired with other libraries for routing, state management, and data fetching.

Why React?

- **Component-based** — UIs are built from small, independent, reusable pieces called components.
- **Declarative** — you describe *what* the UI should look like for a given state, and React figures out *how* to update the DOM to match. This is a major shift from imperative DOM manipulation (`document.getElementById(...).innerHTML = ...`), which is error-prone and hard to scale.
- **Virtual DOM** — React keeps an in-memory representation of the UI (the virtual DOM). When state changes, React computes the difference between the previous and new virtual DOM ("diffing") and updates only the parts of the real DOM that actually changed, rather than re-rendering the whole page.
- **Huge ecosystem** — routing (React Router), state management (Redux, Zustand), styling (Tailwind, styled-components), and a very large community.

Declarative vs Imperative — a quick comparison

```
// Imperative (vanilla JS) — you describe every step
const btn = document.createElement("button");
btn.textContent = "Click me";
btn.addEventListener("click", () => {
  btn.textContent = "Clicked!";
});
document.body.appendChild(btn);
```

```
// Declarative (React) — you describe the end result for each state
function Button() {
  const [clicked, setClicked] = useState(false);
  return (
    <button onClick={() => setClicked(true)}>
      {clicked ? "Clicked!" : "Click me"}
    </button>
  );
}
```

Single Page Applications (SPA)

Traditional websites load a new HTML page from the server on every navigation. An SPA loads a single HTML shell once, and JavaScript (React, plus a router) swaps out content in place as the user navigates — resulting in faster, smoother transitions without full page reloads.

2. Setting Up a React Project

The modern, recommended way to start a React project is with **Vite**, a fast build tool (it replaced Create React App, which is now considered legacy).

```
npm create vite@latest my-app -- --template react
cd my-app
npm install
npm run dev
```

Key project files

File / Folder	Purpose
<code>index.html</code>	The single HTML shell; contains <code><div id="root"></div></code>
<code>src/main.jsx</code>	Entry point — mounts the root <code><App /></code> component into the DOM
<code>src/App.jsx</code>	The root component of your application
<code>package.json</code>	Project dependencies and scripts (<code>dev</code> , <code>build</code> , <code>preview</code>)
<code>vite.config.js</code>	Build tool configuration

```
// src/main.jsx
import React from "react";
import ReactDOM from "react-dom/client";
import App from "./App.jsx";

ReactDOM.createRoot(document.getElementById("root")).render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
);
```

`ReactDOM.createRoot(...).render(...)` is how a React tree gets attached ("mounted") to a real DOM node. `React.StrictMode` is a development-only wrapper that helps catch potential problems (it does not affect production behavior).

3. JSX

JSX (JavaScript XML) is a syntax extension that lets you write HTML-like markup directly inside JavaScript. It is not valid JavaScript by itself — it is compiled (by Babel, under the hood in Vite) into regular `React.createElement()` calls.

```
// What you write:
const element = <h1 className="title">Hello, React!</h1>;

// What it compiles to (conceptually):
const element = React.createElement("h1", { className: "title" }, "Hello, React!");
```

JSX Rules

- Every component must return a **single root element**. Use a Fragment (`<>...</>`) to group elements without adding an extra DOM node.
- Use `className` instead of `class` (since `class` is a reserved JavaScript keyword).
- All tags must be closed — `<img />` , `<br />` , `<input />` are self-closing.
- Embed any JavaScript expression using curly braces `{ }` .

```
function Greeting() {
  const name = "Ranjit";
  const isLoggedIn = true;

  return (
    <>
      <h2>Hello, {name}!</h2>
      <p>{isLoggedIn ? "Welcome back." : "Please log in."}</p>
      <p>2 + 2 = {2 + 2}</p>
    </>
  );
}
```

Important: you can put any *expression* inside `{ }` (things that produce a value), but not *statements* like `if` , `for` , or variable declarations directly. This is why conditional rendering uses ternaries and `&&` rather than plain `if` blocks inline (covered in Section 8).

4. Components

A component is simply a JavaScript function that returns JSX. Modern React uses **function components** almost exclusively (older code may use class components, but these are largely legacy today).

```
function Header() {
  return (
    <header>
      <h1>Fork Infosystems</h1>
      <p>Computer Education Institute</p>
    </header>
  );
}
```

Naming convention: component names must start with a capital letter (PascalCase) — this is how React/JSX distinguishes a custom component (`<Header />`) from a built-in HTML tag (`<header>`).

Composition — components rendering components

Real applications are trees of components. A parent component renders child components, which may render their own children, and so on.

```
function Footer() {
  return <footer><small>© Fork Infosystems</small></footer>;
}

function App() {
  return (
    <div>
      <Header />
      <main>Main content goes here</main>
      <Footer />
    </div>
  );
}
```

The `children` prop

Whatever is placed *between* a component's opening and closing tags is automatically passed to it as a special prop called `children`.

```
function Card({ children }) {
  return <div className="card">{children}</div>;
}

// Usage:
<Card>
  <h3>Course Details</h3>
  <p>This content becomes Card's children.</p>
</Card>
```

This pattern is the foundation of reusable "wrapper" components (modals, cards, layout containers).

5. Props

Props ("properties") are how data flows from a parent component into a child component. Props are **read-only** — a child must never modify its own props directly. Data in React flows in **one direction**: parent → child.

```
function StudentCard({ name, course, marks }) {
  return (
    <div className="card">
      <h4>{name}</h4>
      <p>Course: {course}</p>
      <p style={{ color: marks >= 40 ? "green" : "red" }}>
        {marks >= 40 ? "PASS" : "FAIL"}
      </p>
    </div>
  );
}

function StudentList() {
  return (
    <div>
      <StudentCard name="Aditi Kulkarni" course="React JS" marks={85} />
      <StudentCard name="Rohan Shah" course="React JS" marks={32} />
    </div>
  );
}
```

Props can be any JavaScript value: strings, numbers, booleans, arrays, objects, and even functions (this is essential later for parent-child communication — see Section 12).

Default values can be provided in case a prop is not passed:

```
function Badge({ label = "New" }) {
  return <span className="badge">{label}</span>;
}
```

Key distinction to teach clearly: Props vs State

	Props	State
Owned by	Parent (passed to child)	The component itself
Mutable?	No — read only inside the receiving component	Yes — via its setter function
Purpose	Configure/customize a component from outside	Track data that changes over the component's lifetime

PART B — CORE CONCEPTS

6. State and the useState Hook

Props let data flow in, but components also need to **remember** and **change** data over time — for example, a counter's current value, whether a menu is open, or the text typed into an input. This is what **state** is for.

```
import { useState } from "react";

function Counter() {
  const [count, setCount] = useState(0);
  // count      -> current value
  // setCount    -> function used to update it
  // useState(0) -> 0 is the initial value

  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={() => setCount(count + 1)}>+</button>
      <button onClick={() => setCount(count - 1)}>-</button>
    </div>
  );
}
```

Why you cannot just mutate a variable

```
// WRONG — this will NOT re-render the component
let count = 0;
function increment() { count = count + 1; }

// RIGHT — calling the setter tells React to re-render
const [count, setCount] = useState(0);
function increment() { setCount(count + 1); }
```

React only re-renders a component when its state (or props) changes *through the proper channel* — the setter function returned by `useState`. Directly mutating a plain variable does not notify React that anything changed.

Functional updates

When a new state value depends on the previous one, it is safer to pass a function to the setter rather than referencing the outer variable directly — especially important when multiple updates might be batched together:

```
setCount(prevCount => prevCount + 1);
```

Multiple state variables

A component can hold as many independent `useState` calls as it needs:

```
function LoginForm() {
  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");
  const [showPassword, setShowPassword] = useState(false);
  // ...
}
```

The Rules of Hooks (applies to `useState` and every other hook covered in this document): 1. Only call hooks at the **top level** of a component or custom hook — never inside loops, conditions, or nested functions. 2. Only call hooks from **React function components** or **custom hooks** — never from regular JavaScript functions.

7. Event Handling

React wraps native browser events in a cross-browser-consistent system called **SyntheticEvents**. The API is very close to standard DOM events, with handler names in **camelCase**: `onClick`, `onChange`, `onSubmit`, `onMouseEnter`, `onKeyDown`, and so on.

```
function EventDemo() {
  const [text, setText] = useState("");

  function handleChange(e) {
    setText(e.target.value); // e.target.value = current input value
  }

  function handleSubmit(e) {
    e.preventDefault(); // stops the browser's default full-page-reload form submit
    alert(`Submitted: ${text}`);
  }

  return (
    <form onSubmit={handleSubmit}>
      <input value={text} onChange={handleChange} />
      <button type="submit">Submit</button>
    </form>
  );
}
```

Passing arguments to a handler

```
// Correct – the arrow function is only CALLED when the button is clicked
<button onClick={() => deleteItem(item.id)}>Delete</button>

// WRONG – this calls deleteItem(item.id) immediately during render,
// and passes its RETURN VALUE (often undefined) as the handler
<button onClick={deleteItem(item.id)}>Delete</button>
```

This "calling the function immediately instead of passing a reference" mistake is one of the most common beginner bugs in React.

8. Conditional Rendering

Because JSX only accepts *expressions* (not statements like `if`), React code uses a few standard patterns to render UI conditionally.

Ternary operator — for either/or rendering:

```
{isLoggedIn ? <p>Welcome back!</p> : <p>Please log in.</p>}
```

Logical AND (`&&`) — for rendering something only when a condition is true (renders nothing otherwise):

```
{unreadCount > 0 && <span className="badge">{unreadCount}</span>}
```

Common pitfall: `{count && <Badge/>}` will render a literal `0` on the page if `count` is `0`, because `0` is falsy but is still a renderable value in JSX. Guard against this with `{count > 0 && <Badge/>}` instead.

Early return — useful when a whole component should render differently based on a condition:

```
function UserGreeting({ user }) {  
  if (!user) {  
    return <p>Please log in.</p>;  
  }  
  return <p>Welcome, {user.name}!</p>;  
}
```

9. Lists and Keys

Arrays of data are rendered using the standard JavaScript `.map()` method.

```
const courses = ["React JS", "Node.js", "MongoDB"];

function CourseList() {
  return (
    <ul>
      {courses.map((course, index) => (
        <li key={index}>{course}</li>
      ))}
    </ul>
  );
}
```

Why **key** matters

When React re-renders a list, it uses the **key** of each item to match elements between renders — this lets it correctly reuse, reorder, or remove DOM nodes instead of tearing down and rebuilding the whole list on every change. Without stable keys, React can mismatch state between list items, causing subtle bugs (wrong input values ending up attached to the wrong row, animations firing on the wrong element, and so on).

Best practice: use a stable, unique identifier from your data (**item.id**) as the key — **not** the array index — whenever the list can be reordered, filtered, or have items inserted/removed in the middle.

```
const students = [
  { id: 101, name: "Aditi" },
  { id: 102, name: "Rohan" },
];

{students.map(student => (
  <li key={student.id}>{student.name}</li>
))}
```

Array index as a key is only acceptable for lists that are static and never reordered or filtered.

Combining **.filter()** and **.map()**

```
function ActiveTasks({ tasks }) {
  return (
    <ul>
      {tasks
        .filter(task => !task.done)
        .map(task => <li key={task.id}>{task.title}</li>)}
    </ul>
  );
}
```

10. Forms and Controlled Components

In HTML, form elements (`<input>`, `<textarea>`, `<select>`) keep their own internal state in the DOM. In React, the recommended pattern is a **controlled component** — the input's value is driven entirely by React state, and every keystroke updates that state via `onChange`.

```
function RegistrationForm() {
  const [form, setForm] = useState({ name: "", email: "" });

  function handleChange(e) {
    const { name, value } = e.target;
    setForm(prev => ({ ...prev, [name]: value }));
    // ONE handler for ALL fields, using the input's "name" attribute
  }

  function handleSubmit(e) {
    e.preventDefault();
    console.log(form);
  }

  return (
    <form onSubmit={handleSubmit}>
      <input name="name" value={form.name} onChange={handleChange} />
      <input name="email" value={form.email} onChange={handleChange} />
      <button type="submit">Register</button>
    </form>
  );
}
```

Because the input's `value` always equals `form.name` / `form.email` from state, React (not the DOM) is the "single source of truth" for the form's data at every moment — this makes it trivial to validate, transform, or reset the data.

Checkboxes and selects

```
<input
  type="checkbox"
  checked={form.subscribe}
  onChange={(e) => setForm(prev => ({ ...prev, subscribe: e.target.checked })))}
/>

<select name="course" value={form.course} onChange={handleChange}>
  <option>React JS</option>
  <option>Node.js</option>
</select>
```

Note: checkboxes use `e.target.checked` (a boolean), while text inputs and selects use `e.target.value` (a string).

Basic validation

```
function validate(form) {  
  const errors = {};  
  if (!form.name.trim()) errors.name = "Name is required";  
  if (!/^[\s@]+@[\s@]+\.[\s@]+$/ .test(form.email)) {  
    errors.email = "Enter a valid email";  
  }  
  return errors;  
}
```

PART C — INTERMEDIATE

11. useEffect and Side Effects

Rendering in React is meant to be a "pure" calculation: given the same props/state, a component should produce the same JSX. Anything that reaches *outside* that — timers, subscriptions, manually touching the DOM, fetching data from a server — is called a **side effect**, and belongs inside the `useEffect` hook.

```
import { useState, useEffect } from "react";

function LiveClock() {
  const [time, setTime] = useState(new Date());

  useEffect(() => {
    const intervalId = setInterval(() => setTime(new Date()), 1000);

    // Cleanup function: runs when the component unmounts,
    // or before the effect runs again — prevents leaked timers/listeners.
    return () => clearInterval(intervalId);
  }, []); // empty array = run once, when the component first mounts

  return <p>{time.toLocaleTimeString()}</p>;
}
```

The dependency array — three patterns

Dependency array	Behavior
(omitted entirely)	Effect runs after every render
<code>[]</code>	Effect runs once , when the component mounts
<code>[a, b]</code>	Effect runs on mount, and again whenever <code>a</code> or <code>b</code> changes

```
useEffect(() => {
  document.title = `You clicked ${count} times`;
}, [count]); // re-runs only when `count` changes
```

Why cleanup matters

Without a cleanup function, timers, subscriptions, and event listeners keep running even after a component is removed from the page — a classic source of memory leaks and "why is this running twice" bugs.

```
useEffect(() => {
  function handleResize() { console.log(window.innerWidth); }
  window.addEventListener("resize", handleResize);
  return () => window.removeEventListener("resize", handleResize);
}, []);
```

12. Component Composition and Lifting State Up

As applications grow, two **sibling** components often need to share and stay in sync with the same piece of data (for example, a product list and a cart summary). Since data can only flow *down* through props, the solution is to move ("lift") the shared state up to their closest common parent, which then passes the data down as props and passes update functions down as callback props.

```
function ProductList({ onAddToCart }) {
  const products = [{ id: 1, name: "Keyboard", price: 799 }];
  return (
    <div>
      {products.map(p => (
        <button key={p.id} onClick={() => onAddToCart(p)}>
          Add {p.name}
        </button>
      ))}
    </div>
  );
}

function CartSummary({ items }) {
  const total = items.reduce((sum, item) => sum + item.price, 0);
  return <p>Cart total: ₹{total}</p>;
}

function ShoppingCart() {
  const [cartItems, setCartItems] = useState([]); // state lives in the common parent

  function addToCart(product) {
    setCartItems(prev => [...prev, product]);
  }

  return (
    <div>
      <ProductList onAddToCart={addToCart} />
      <CartSummary items={cartItems} />
    </div>
  );
}
```

`ProductList` and `CartSummary` never talk to each other directly — they both talk to their shared parent, `ShoppingCart`, which owns the single source of truth. This pattern scales well for small-to-

medium apps; for larger apps, Context (Section 14) or a dedicated state library (Section 21) becomes more practical.

13. React Router

Because a React app is a Single Page Application, "pages" are not separate HTML files — they are different components rendered based on the current URL. **React Router** (`react-router-dom`) is the standard library for this.

```
npm install react-router-dom
```

```
import { BrowserRouter, Routes, Route, Link } from "react-router-dom";

function Home() { return <h2>Home Page</h2>; }
function About() { return <h2>About Us</h2>; }

function App() {
  return (
    <BrowserRouter>
      <nav>
        <Link to="/">Home</Link>
        <Link to="/about">About</Link>
      </nav>
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/about" element={<About />} />
        <Route path="*" element={<h2>404 - Not Found</h2>} />
      </Routes>
    </BrowserRouter>
  );
}
```

`<Link>` is used instead of a plain `<a>` tag — it updates the URL and swaps the rendered component **without** a full browser page reload.

Dynamic route parameters

```
import { useParams, useNavigate } from "react-router-dom";

<Route path="/courses/:courseId" element={<CourseDetail />} />

function CourseDetail() {
  const { courseId } = useParams(); // read the :courseId segment from the URL
  const navigate = useNavigate();   // navigate programmatically (e.g. after an action)

  return (
    <div>
      <p>Viewing course: {courseId}</p>
      <button onClick={() => navigate(-1)}>Go Back</button>
    </div>
  );
}
```

Nested routes with `<Outlet />`

Nested routes let a shared layout (e.g., a sidebar) stay on screen while only the inner content changes as the URL changes:

```
import { Outlet } from "react-router-dom";

function CourseLayout() {
  return (
    <div className="layout">
      <aside>{/* sidebar links */}</aside>
      <Outlet /> {/* the matched child route renders here */}
    </div>
  );
}

<Route path="/courses" element={<CourseLayout />}>
  <Route index element={<CoursesHome />} />
  <Route path=":courseId" element={<CourseDetail />} />
</Route>
```

14. Context API

Passing props down through many layers of components that don't actually use the data themselves — just to reach a deeply nested child — is called **prop drilling**. The Context API solves this by letting any descendant read a value directly, without it being passed explicitly at every level.

```

import { createContext, useContext, useState } from "react";

const ThemeContext = createContext(); // 1. Create the context

function ThemeProvider({ children }) {
  const [theme, setTheme] = useState("light");
  const toggleTheme = () => setTheme(t => (t === "light" ? "dark" : "light"));

  return (
    // 2. Wrap the part of the tree that needs access, providing a value
    <ThemeContext.Provider value={{ theme, toggleTheme }}>
      {children}
    </ThemeContext.Provider>
  );
}

function ThemeButton() {
  // 3. Any descendant can read the value directly – no props needed
  const { theme, toggleTheme } = useContext(ThemeContext);
  return <button onClick={toggleTheme}>Current theme: {theme}</button>;
}

function App() {
  return (
    <ThemeProvider>
      <SomeDeeplyNestedComponent>
        <ThemeButton />
      </SomeDeeplyNestedComponent>
    </ThemeProvider>
  );
}

```

When to use Context: genuinely global-ish data — the current theme, the logged-in user, the selected language. **When not to:** Context is not a full state-management replacement for everything; for data that's only shared between a few nearby components, plain props (Section 12) are simpler and easier to trace.

PART D — ADVANCED

15. Custom Hooks

A **custom hook** is simply a function whose name starts with `use`, built out of other hooks, that lets you extract and reuse stateful logic across multiple components — without repeating the same `useState` / `useEffect` code everywhere.

```
import { useState, useEffect } from "react";

function useLocalStorage(key, initialValue) {
  const [value, setValue] = useState(() => {
    const stored = localStorage.getItem(key);
    return stored !== null ? JSON.parse(stored) : initialValue;
  });

  useEffect(() => {
    localStorage.setItem(key, JSON.stringify(value));
  }, [key, value]);

  return [value, setValue];
}

// Usage — works exactly like useState, but now persists automatically:
function NotesApp() {
  const [note, setNote] = useLocalStorage("note", "");
  return <textarea value={note} onChange={e => setNote(e.target.value)} />;
}
```

Custom hooks follow the same Rules of Hooks as built-in ones (Section 6) — they can call other hooks internally, but only at the top level.

useRef — a companion tool for custom hooks and DOM access

`useRef` creates a mutable value that persists across renders **without** causing a re-render when it changes. Its two main uses: directly referencing a DOM node, and storing a mutable value (like an interval ID) that isn't part of the visible UI.

```
import { useRef } from "react";

function FocusInput() {
  const inputRef = useRef(null);
  return (
    <>
      <input ref={inputRef} />
      <button onClick={() => inputRef.current.focus()}>Focus</button>
    </>
  );
}
```

Rule of thumb: if changing the value should update what's on screen, use `useState`. If it shouldn't (and doesn't need to), use `useRef`.

16. useReducer

`useState` works well for simple, independent values. When state updates involve **multiple related pieces of logic** (add/toggle/delete items, for instance), `useReducer` often keeps the code more organized by centralizing all the update logic in one place — a **reducer function**.

```

import { useReducer } from "react";

function tasksReducer(state, action) {
  switch (action.type) {
    case "ADD":
      return [...state, { id: Date.now(), title: action.title, done: false }];
    case "TOGGLE":
      return state.map(t => t.id === action.id ? { ...t, done: !t.done } : t);
    case "DELETE":
      return state.filter(t => t.id !== action.id);
    default:
      return state;
  }
}

function TaskManager() {
  const [tasks, dispatch] = useReducer(tasksReducer, []);

  return (
    <>
      <button onClick={() => dispatch({ type: "ADD", title: "New task" })}>
        Add Task
      </button>
      {tasks.map(t => (
        <div key={t.id}>
          <span onClick={() => dispatch({ type: "TOGGLE", id: t.id })}>
            {t.done ? " " : " "} {t.title}
          </span>
          <button onClick={() => dispatch({ type: "DELETE", id: t.id })}>x</button>
        </div>
      ))}
    </>
  );
}

```

Instead of calling several different setter functions scattered throughout event handlers, every possible state change is described by a plain **action object** (`{ type: "ADD", ... }`) and handled in exactly one place — the reducer. This makes complex state transitions easier to trace, test, and debug.

17. Performance Optimization

By default, when a component re-renders, all of its child components re-render too — even if their own props haven't changed. For most apps this is fast enough to never matter. For components that are expensive to render, or that re-render very frequently, React provides a few tools to skip unnecessary work.

React.memo — skips re-rendering a component if its props haven't changed:

```
const ExpensiveRow = React.memo(function ExpensiveRow({ data }) {
  console.log("Rendering row", data.id);
  return <tr><td>{data.name}</td></tr>;
});
```

useMemo — caches ("memoizes") the *result* of an expensive calculation, recomputing only when its dependencies change:

```
const sortedList = useMemo(() => {
  return [...items].sort((a, b) => a.price - b.price);
}, [items]);
```

useCallback — caches a *function definition* itself, so that a child receiving it as a prop doesn't see a "new" function on every render (important when combined with **React.memo**):

```
const handleAdd = useCallback((item) => {
  setItems(prev => [...prev, item]);
}, []);
```

Important teaching point: these tools are optimizations, not defaults. Reaching for **useMemo** / **useCallback** / **React.memo** everywhere adds complexity for little benefit in small apps — they matter most in lists with hundreds/thousands of rows, expensive computations, or components that are proven (via profiling) to be a bottleneck.

18. Connecting to APIs

Most real applications need to fetch data from a server. The built-in **fetch()** API combined with **async/await** inside **useEffect** is the standard pattern.

```

import { useState, useEffect } from "react";

function UserList() {
  const [users, setUsers] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    let cancelled = false;

    async function fetchUsers() {
      try {
        setLoading(true);
        const res = await fetch("https://jsonplaceholder.typicode.com/users");
        if (!res.ok) throw new Error(`HTTP ${res.status}`);
        const data = await res.json();
        if (!cancelled) setUsers(data);
      } catch (err) {
        if (!cancelled) setError(err.message);
      } finally {
        if (!cancelled) setLoading(false);
      }
    }

    fetchUsers();
    return () => { cancelled = true; }; // avoid setting state after unmount
  }, []);

  if (loading) return <p>Loading...</p>;
  if (error) return <p>Error: {error}</p>;

  return <ul>{users.map(u => <li key={u.id}>{u.name}</li>)}</ul>;
}

```

Every data-fetching component should generally handle **three states**: loading, error, and success — skipping any of these leads to broken-looking UIs (blank screens on error, no feedback while waiting, and so on).

Note on `useEffect` and `async`: the effect callback itself cannot be declared `async` directly (`useEffect(async () => {...})` is invalid), because `useEffect` expects its callback to return either nothing or a cleanup function — not a Promise. The standard workaround, shown above, is to define an `async` function inside the effect and call it immediately.

POST requests (sending data)

```

async function createUser(userData) {
  const res = await fetch("/api/users", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(userData),
  });
  return res.json();
}

```

`axios` is a popular third-party alternative to `fetch` (automatic JSON parsing, request/response interceptors, easier error handling) but is not required — `fetch` is sufficient for most applications and is built into every modern browser.

19. Error Boundaries

An uncaught JavaScript error inside any component, by default, unmounts the **entire** React component tree, resulting in a blank page. An **Error Boundary** is a component that catches errors in its child tree and renders a fallback UI instead of crashing the whole app.

```

import { Component } from "react";

class ErrorBoundary extends Component {
  state = { hasError: false };

  static getDerivedStateFromError() {
    return { hasError: true };
  }

  componentDidCatch(error, info) {
    console.error("Caught by ErrorBoundary:", error, info);
  }

  render() {
    if (this.state.hasError) {
      return <h2>Something went wrong. Please refresh the page.</h2>;
    }
    return this.props.children;
  }
}

// Usage:
<ErrorBoundary>
  <Dashboard />
</ErrorBoundary>

```

Error Boundaries are one of the few remaining places a class component is still required — there is currently no hook-based equivalent for catching render-time errors. Note that Error Boundaries only catch errors during rendering, in lifecycle methods, and in constructors of the tree below them — they

do **not** catch errors inside event handlers (those should be handled with a normal `try/catch`) or inside asynchronous code like `setTimeout` or `fetch` callbacks.

20. Code Splitting and Lazy Loading

By default, a bundler like Vite packages the entire application into one (or a few) JavaScript files, which the browser downloads before it can render anything. As an app grows, this bundle grows too, slowing down the initial page load. **Code splitting** breaks the bundle into smaller chunks that are only downloaded when actually needed.

```
import { lazy, Suspense } from "react";

// Instead of a normal top-level import, load this component's code
// only when it is actually about to be rendered:
const AdminPanel = lazy(() => import("./AdminPanel.jsx"));

function App() {
  return (
    <Suspense fallback={<p>Loading...</p>}>
      <AdminPanel />
    </Suspense>
  );
}
```

`<Suspense>` shows its `fallback` content while the lazy component's code is still being downloaded. This is especially valuable combined with routing — each page/route can be its own lazily-loaded chunk, so users only download the code for the page they're actually visiting.

21. State Management Beyond React

For small and medium applications, `useState`, `useReducer`, and Context (Sections 6, 16, 14) are usually sufficient. As an application grows very large — many features, deeply nested components, frequent cross-cutting state updates — dedicated state management libraries can help keep things organized.

Library	Notes
Redux (with Redux Toolkit)	The most established option; centralizes all app state in a single store with a strict, predictable update pattern (very similar in spirit to <code>useReducer</code> , but app-wide). Has a steeper learning curve.
Zustand	A lightweight, minimal-boilerplate alternative — a good middle ground between plain Context and full Redux for small-to-medium apps.

Library	Notes
React Query / TanStack Query	Not a general state manager — specifically for <i>server</i> state (API data): caching, background refetching, loading/error states handled automatically, reducing a lot of the manual <code>useEffect</code> fetching code from Section 18.

Guidance for students: don't reach for these by default. Learn to recognize the pain points first (state genuinely needed across many unrelated components, or a lot of repetitive data-fetching boilerplate) — these tools solve *that* pain, and adopting them before the pain exists usually just adds unnecessary complexity.

22. Deployment and Next Steps

Building for production

```
npm run build
```

This produces an optimized, minified `dist/` folder — this is what actually gets deployed, not the raw source code.

Deploying (Netlify / Vercel — both free-tier friendly)

1. Push the project to a GitHub repository.
2. Connect the repository on Netlify or Vercel.
3. Set the build command to `npm run build` and the publish/output directory to `dist`.
4. Every subsequent push to the connected branch automatically triggers a new deployment.

Where to go from here

- **TypeScript** — adds static typing on top of JavaScript/React; catches many bugs before runtime and is increasingly the default in professional React codebases.
- **Next.js** — a React framework built on top of React itself, adding server-side rendering, file-based routing, and built-in API routes — commonly used for production applications that need strong SEO or faster initial page loads.
- **Testing** — React Testing Library (component-level tests) and tools like Vitest/Jest for a solid testing workflow.
- **Component libraries** — Tailwind CSS, shadcn/ui, Material UI, or Chakra UI to speed up building polished interfaces without writing all styling from scratch.

End of Theory Notes.